

A report and documentation of the ACSL-SL Bridge

Hrishiraj Mandal

April 29, 2026

1 Introduction

Specification languages are widely used to describe and verify the functional correctness of a program. In this project, the focus is to provide a translation mechanism between Separation Logic(SL) and ACSL. These languages operate at different levels of expressiveness in that SL is well-suited for writing concise specifications. However, ACSL is the primary annotation language supported by widely-used prover tools such as Frama-C. The translation of SL into ACSL is non-trivial because the two languages differ in terms of syntax and how much is written to express the same ideas.

2 Scope of work

2.1 Scope of translation

The current scope of work provides the translation of SL to ACSL. A future work can now reliably translate ACSL to SL as then there exists a means for a conversion of the generated SL to be translated into ACSL to be checked on Frama-C.

2.2 Scope of syntax translated

The following are samples of the main classes of programs that the translator supports: (i) pointer (spatial) notation, (ii) case notation, and (iii) loop-invariant notation.

pointer (spatial) notation

```
/*@[SL]
req a $\rightarrow$ int*(u);
ens a $\rightarrow$ int*(u);
*/
```

ACSL (output)

```
/*@
  requires \valid(a);
  assigns *a;
  ensures *a == \old(*a);
*/
```

Case notation

```
/*@[SL]
req p$\rightarrow$int*(a) ** q$\rightarrow$int*(b);
case {
  a>=b ==> ens p$\rightarrow$int*(a+1) && q$\rightarrow$int*(b);
  a<b ==> ens p$\rightarrow$int*(a) && q$\rightarrow$int*(b+1);
};
*/
```

ACSL (output)

```
/*@
  requires p != q && \valid(p) && \valid(q);
  assigns *p, *q;
  behavior case1:
    assumes *p >= *q;
    ensures *p == \old(*p) + 1 && *q == \old(*q);
  behavior case2:
    assumes *p < *q;
    ensures *p == \old(*p) && *q == \old(*q) + 1;
  complete behaviors;
  disjoint behaviors;
*/
```

Loop notation Prime notation in SL (e.g., i' , b') induces ACSL references to $\text{\texttt{\textbackslash at}(\text{\textbackslash cdot}, \text{LoopEntry})}$ in the loop invariant.

SL (input)

```
req i<=10 && Term[10-i];
ens i'==10 && b'==b+(i'-i);
```

ACSL (output)

```
/*@
  loop invariant i <= 10;
  loop invariant b == \at(b, LoopEntry) + (i - \at(i, LoopEntry));
  loop assigns b, i;
  loop variant 10 - i;
*/
```

3 User Documentation

Users are able to automatically translate an input c file containing SL annotation and run the output c file containing ACSL annotation on frama-c.

The batch files are found at the project root labelled `sl_to_acsl.sh` `sl_to_acsl.bat`.

The batch file for the execution with frama-c command line is as follows:

```
./sl_to_acsl.sh <file-path>
```

The batch file for the execution with frama-c GUI is as follows:

```
./sl_to_acsl.sh --gui <file-path>
```

4 Reasoning of Architecture

4.1 Architecture of translator

Given an input of SL, its translation to ACSL follows a pipeline of: parsing and lexing following the given Abstract Syntax Tree(AST) → a conversion to a core representation to compress the variety of ways representing the same concept → a conversion to ACSL → printing the ACSL output. The aforementioned pipeline defines the main SL-ACSL translation logic.

4.2 Architecture of translator and verification

For convenience of verifying the SL-ACSL translation, a batch file is written that integrates the SL-ACSL translator with Frama-C. Precisely, the pipeline of the batch file is as follows: an input c file containing SL annotation → SL-ACSL translation → an output c file containing ACSL annotation → verification on Frama-c on command line or GUI.

4.3 High-level overview of translating different classes

4.3.1 Translating the pointer

Translation of the pointer involves distinguishing the prime ($'$) and $old(@)$ notation. In *sl_to_core*, they are both translated to the same core representation.

4.3.2 Translating the case specifications

Case specifications are given general names to distinguish the different branches while utilising the translation of the pointer to translate the contents of each branch.

4.3.3 Translating the loop contract

The translation from loop contracts to loop invariants follows the key insight that a contract specifies the entire effect of a loop as a relation between its entry and exit states while an invariant must hold at every intermediate iteration. To

bridge this gap, the translation systematically “lifts” the contract into an invariant by remembering the loop-entry state and enforcing that the current state remains compatible with the original summary. This is achieved by anchoring invariants to the loop-entry snapshot (e.g., using *at(..., LoopEntry)*), splitting the loop’s working domain into a processed prefix and an unprocessed suffix. The processed portion is required to already satisfy the contract’s postcondition relative to the entry state, while the unprocessed portion is constrained to remain unchanged. This structure ensures inductiveness because each iteration moves exactly one element from the unchanged region into the processed region while preserving the contract relation. Safety conditions from the contract become invariant clauses.

5 Developer Documentation: Abstract Syntax Tree and its printer

The abstract syntax tree defining SL, Core and ACSL are found in the folder labelled `ast`. Correspondingly, their printers are found in the folder labelled `ast_printer`. `ast_printer` provides a trivial implementation of printing the corresponding ast.

6 Developer Documentation: `sl_to_core`

This module translates the parsed SL AST (`Sl_ast`) into an intermediate Core specification AST (`Core`). The Core layer normalizes syntactic sugar (e.g. `\old`, `'`, etc.), separates concerns (heap vs pure), and computes contract structure (function contract vs loop contract), to enable simpler conversion to ACSL.

6.1 Module `PairOrd` and `PairSet`

`PairOrd.compare` : $(\text{string} * \text{string}) \rightarrow (\text{string} * \text{string}) \rightarrow \text{int}$. **Input:** two ordered pairs of pointer variable names.

Output: a total order (lexicographic) used to build sets of pairs.

Why: separation constraints produce sets of unordered pointer-inequality pairs. Normalizing to a canonical order and storing them in a set avoids duplicates and ensures determinism.

6.2 Module `Util`

This module provides small combinators for building Core predicates in a normalized form.

`list.filter_map` : $(\text{'a} \rightarrow \text{'b option}) \rightarrow \text{'a list} \rightarrow \text{'b list}$. **Input:** mapping function that may discard elements, and a list.

Output: list of all produced `Some` values in original order.

Why: used throughout extraction passes where only some nodes contribute (e.g., heap bases inside `SSep` groups).

`flatten_and : Core.predicate list → Core.predicate list.` **Input:** a list of predicates.

Output: same list but with `PTrue` removed and nested `PAnd` flattened.

Why: canonicalizes conjunctions to avoid deep trees and redundant identities, making later pattern matching and printing stable.

`flatten_or : Core.predicate list → Core.predicate list.` **Input:** a list of predicates.

Output: list with `PFalse` removed and nested `POr` flattened.

Why: same as `flatten_and`, but for disjunctions.

`p_and : Core.predicate list → Core.predicate.` **Input:** list of predicates.

Output: `PTrue` for empty, the element for singleton, otherwise `PAnd`.

Why: provides a uniform construction API for conjunctions while preserving a normalized shape.

`p_or : Core.predicate list → Core.predicate.` **Input:** list of predicates.

Output: `PFalse` for empty, singleton preserved, otherwise `POr`.

Why: analogous normalized construction for disjunctions.

`p_atom : Core.atom → Core.predicate.` **Input:** an atom.

Output: `PAtom atom`.

Why: makes predicate construction uniform and readable.

`mk_rel : Core.rel → Core.term → Core.term → Core.predicate.` **Input:** relation and two terms.

Output: atomic predicate `ARel(r,t1,t2)`.

Why: standard constructor used by many inference passes (e.g., inferred pointer inequalities).

`mk_eq : Core.term → Core.term → Core.predicate.` **Input:** two terms.

Output: equality predicate.

Why: equality is the most common relational form (heaplet equality, accumulator invariants).

6.3 Module Traverse

This module implements structural folds over SL expressions and SL formulas.

fold_expr : $f : ('a \rightarrow \text{expr} \rightarrow 'a) \rightarrow 'a \rightarrow \text{expr} \rightarrow 'a$. **Input**: an accumulator, a visitor function f , and an SL expression.

Output: folded accumulator.

Why: many analyses need “scan the tree and collect X” (variables, post-writes, alias bases) without rewriting.

fold_sl : $f_sl : ('a \rightarrow \text{sl} \rightarrow 'a) \rightarrow f_expr : ('a \rightarrow \text{expr} \rightarrow 'a) \rightarrow 'a \rightarrow \text{sl} \rightarrow 'a$. **Input**: two visitors (f_sl for SL nodes and f_expr for expressions), accumulator, SL formula.

Output: folded accumulator.

Why: separates traversal of the logical structure from traversal of embedded expressions, enabling flexible collectors: e.g., collect heaplets via f_sl while ignoring expressions, or vice versa.

6.4 Module Binder

core_ty_of_sort_opt : $\text{sort option} \rightarrow \text{string option}$. **Input**: SL binder sort (optional).

Output: optional Core type string.

Why: Core binders carry optional surface type names for printing and downstream target typing. Pointer sorts map to **None** because pointer-typed quantifiers are not directly emitted as a scalar type in Core (pointer-ness is encoded at term level).

binders_of_sl : $(\text{ident} * \text{sort option}) \text{ list} \rightarrow \text{Core.binder list}$. **Input**: SL binders.

Output: Core binders with b_name and optional b_ty .

Why: quantifiers are preserved through translation; Core provides a uniform binder representation.

6.5 Module Op

arith_of_binop : $\text{Sl_ast.binop} \rightarrow \text{Core.arith_op option}$. **Input**: SL binary operator.

Output: corresponding Core arithmetic operator, if applicable.

Why: Core separates arithmetic from relational logic. Non-arithmetic binops fall back to a generic encoding.

rel_of_binop : $\text{Sl_ast.binop} \rightarrow \text{Core.rel option}$. **Input**: SL binary operator.

Output: corresponding Core relation, if applicable.

Why: comparisons become Core relational atoms; other expressions are treated as boolean-valued terms.

6.6 Module Expr

`term_of_expr` : `Core.spec_kind` $\rightarrow$ `Core.phase` $\rightarrow$ `Sl_ast.expr` $\rightarrow$ `Core.term`.

Input:

- `kind`: whether this spec is a `FunctionContract` or `LoopContract`,
- `default_phase`: the phase used for unannotated variables/dereferences (`Pre`, `Post`, or `LoopEntry`),
- SL expression.

Output: a Core term with explicit phase annotations on variables and heap reads.

Why: Core makes temporal reasoning explicit:

- `EVar x` becomes `TVar(default_phase, x)`.
- `EOld e` rewrites to phase `Pre` for function contracts, and `LoopEntry` for loop contracts.
- `EPost e` forces phase `Post`.

This uniform phasing allows downstream translation to ACSL `\old` and `\at(..., LoopEntry)`.

Memory operations and the `inner_phase_for_mem_op` rule. **Input:** a phase `ph`.

Output: an “inner” phase for the base/index subterms of a dereference/index.

Why: when computing a post-state heap read (e.g., array indexing under `Post`), the base and index expressions should often be evaluated at loop-entry rather than at post to avoid generating self-referential loop-entry constraints. The implementation maps `LoopEntry` to `Post` for inner subterms; otherwise it preserves the phase.

Special cases in dereference translation. **Input:** `EDeref` patterns.

Output: one of `THeap`, `TLoad`, or `TIndex`.

Why:

- `EDeref(EVar p)` is the canonical heap-cell read: `THeap(phase, p)`.
- `EDeref(EBinop(BAdd, base, idx))` becomes array indexing: `TIndex(phase, base, idx)`.
- Other deref forms become `TLoad`, preserving a general memory-load construct.

6.7 Module Pred

`pred_of_cmp_expr` : `Core.spec_kind` $\rightarrow$ `Core.phase` $\rightarrow$ `Sl_ast.expr` $\rightarrow$ `Core.predicate`.

Input: kind, default phase, and an expression intended as a boolean condition.

Output: either a relational atom (if the expression is a comparison) or a boolean predicate wrapper.

Why: SL pure conditions use expressions; Core normalizes comparisons into `ARel` atoms for simpler reasoning. Non-comparison expressions are treated as boolean-valued terms via `APred("bool", ...)`.

`pred_of_sl_with_phase` : `Core.spec_kind` $\rightarrow$ `Core.phase` $\rightarrow$ `Sl_ast.sl` $\rightarrow$ `Core.predicate`. **Input:** kind, phase, and an SL formula.

Output: Core predicate representing the *pure* logical content of the SL formula.

Why: heaplets (`SHeap`) and `SEmp` do not directly become logical predicates here; they are handled by separate heap-specific passes (validity, assigns, ensures from heaplets). This function thus extracts the pure skeleton: `SAnd/SSep` map to conjunction, `SOr` to disjunction, quantifiers become Core quantifiers.

`pred_of_sl` : `Core.spec_kind` $\rightarrow$ `Sl_ast.sl` $\rightarrow$ `Core.predicate`. **Input:** kind and SL formula.

Output: Core predicate using default phase `Pre`.

Why: contracts default to pre-state unless explicitly shifted.

6.8 Module Block

`extract_req_ens_var` : `Sl_ast.block` $\rightarrow$ (`sl option` * `sl option` * `expr option`). **Input:** a behavior block (list of clauses) containing `CReq`, `CEns`, and optional `CVar`.

Output: the extracted `req`, `ens`, and `variant` (each optional).

Why: SL behaviors are structurally flat; translation needs direct access to these three components for analysis.

6.9 Module Rewrite

This module implements semantic-preserving rewrites needed to normalize user-facing sugar into a form that later passes assume.

`force_old_unprimed_heap_reads_if_post_exists` : `sl` $\rightarrow$ `sl`. **Input:** an SL formula (typically an ensures).

Output: a rewritten SL formula where unannotated heap reads are wrapped in `EOld` if any post-state heap write exists.

Why: in patterns like search-and-replace, the ensures references both pre- and post-state heap contents. If there is any explicit post heap write (e.g., `array[j]'`), then unprimed reads in the ensures are intended as *old* reads. This rewrite enforces that convention to avoid mixing states implicitly.

rewrite_result : `string` $\rightarrow$ `sl` $\rightarrow$ `sl`. **Input:** the result variable name from `ens[r]`, and an SL formula.

Output: same formula with occurrences of that variable rewritten to `EResult`.

Why: Core has a distinguished `\result` term; normalizing early prevents ad-hoc handling later.

pre_value_to_loc_map : `(string * string) list` $\rightarrow$ `string StringMap.t`.

Input: list of heap point-to atoms in the form `(loc,valueVar)`.

Output: map from `valueVar` to `loc`.

Why: SL often uses value variables (e.g., `a`, `b`) as stand-ins for `*p`, `*q` in pure parts. This map enables rewriting those value variables back into dereferences.

rewrite_value_vars_with_pre_map : `pre_map` $\rightarrow$ `sl` $\rightarrow$ `sl`. **Input:** a value $\rightarrow$ location map derived from pre heaplets, and an SL formula.

Output: SL formula where occurrences of value variables are replaced by dereferences of the corresponding locations, wrapped in `EOld` unless already under `EOld`.

Why: this is the key bridge between the “symbolic heap” view (`p $\rightarrow$ int*(a)`) and pure reasoning: `a $\geq$ b` becomes `*p  $\geq$  *q` (with appropriate old/post annotation).

rewrite_range_aliases : `alias_map` $\rightarrow$ `sl` $\rightarrow$ `sl`. **Input:** map from alias name to base pointer name, and an SL formula.

Output: SL formula with aliases rewritten to their base pointers (including simple `a + idx` patterns).

Why: array-range sugar introduces aliases such as `arr` / `narr`. Downstream translation expects a single base name for validity, assigns, and indexing.

6.10 Module Heap

This module extracts heap-relevant atoms and alias information from SL formulas.

Types `pt_atom`, `pt_atom_any`, `range_atom`. **Purpose:** store normalized summaries of heaplets:

- `pt_atom`: point-to with both `loc` and `value` as variables,
- `pt_atom_any`: point-to with arbitrary value expression,
- `range_atom`: array segment footprint `base`, `lo`, `hi`, `mode`.

Why: these summaries allow separate inference passes (validity, assigns, ensures) without re-traversing raw SL repeatedly.

union_alias_maps : `map → map → map`. **Input:** two alias maps.
Output: union map preferring the left binding on conflicts.
Why: aliases can be declared in `requires`, `ensures`, and `assumes`; union yields one consistent normalization map.

collect_range_aliases : `sl → string StringMap.t`. **Input:** SL formula.
Output: map from alias names to “real” bases.
Why: range heaplets (`HRange`) may introduce `alias=Some a`. The implementation prefers mapping the non-real base to the real base when both appear, stabilizing normalization when the user writes either base or alias.

expand_n_aliases : `map → map`. **Input:** alias map.
Output: alias map extended with `n` prefixed variants (unless alias already begins with `n`).
Why: the test suite uses `arr/narr` conventions. This heuristic makes `narr` resolve to the same base as `arr` automatically.

collect_pt_atoms : `sl → pt_atom list`. **Input:** SL formula.
Output: all point-to heaplets of the restricted form `p→*(v)` where both sides are variables.
Why: these atoms drive the `value→location` rewrite and pointer-set extraction.

collect_pt_atoms_any : `sl → pt_atom_any list`. **Input:** SL formula.
Output: all point-to heaplets with arbitrary value expressions.
Why: ensures construction for heaplets needs full expressions (e.g., `a+1`).

collect_range_atoms : `sl → range_atom list`. **Input:** SL formula.
Output: extracted range footprints from `HRange` and recognized predicate-form ranges (`HPred`).
Why: range footprints determine `\valid/\valid_read` obligations and array-range assigns. The function recognizes both explicit `HRange` and a predicate encoding by inspecting the first argument as a base.

ptrs_from_pt_atoms : `pt_atom list → StringSet.t`. **Input:** point-to atoms.
Output: set of location variables.
Why: pointer locations define required validity and heap assigns for scalar cells.

pre_pairs_for_map : `pt_atom list → (string * string) list`. **Input:** point-to atoms.
Output: list of `(loc,valueVar)` pairs.
Why: feeds `Rewrite.pre_value_to_loc_map`.

6.11 Module Desugar

desugar_expr : `pre_alias` $\rightarrow$ `post_alias` $\rightarrow$ `expr` $\rightarrow$ `expr`. **Input:** alias maps and an SL expression.

Output: expression where dereferences through aliases are wrapped as `EOld` (pre alias) or `EPost` (post alias).

Why: array-alias notation (`arr`, `narr`) represents pre/post snapshots. This pass makes the temporal intent explicit before Core phasing is computed.

desugar_sl : `pre_alias` $\rightarrow$ `post_alias` $\rightarrow$ `sl` $\rightarrow$ `sl`. **Input:** alias maps and SL formula.

Output: formula with `desugar_expr` applied everywhere.

Why: ensures all nested occurrences are normalized uniformly.

6.12 Module Sugar

collect_heap_equalities_from_pure : `sl` $\rightarrow$ (`string` * `string`) list. **Input:** SL formula.

Output: list of pointer-pairs (`a`,`b`) extracted from pure equalities that relate heap reads across phases, e.g., `(*a)' == (*b)` or `(*a) == \old(*b)`.

Why: the swap tests express heap updates as pure equalities rather than explicit heaplets. These equalities should contribute to: (1) pointer discovery, and (2) ensures construction as heap equalities.

6.13 Module Collect

primed_scalar_vars : `sl` $\rightarrow$ `StringSet.t`. **Input:** SL formula.

Output: variables appearing as `EPost(EVar x)`.

Why: loop assigns must include scalar variables written in the post-state.

vars_in_expr : `expr` $\rightarrow$ `StringSet.t`. **Input:** an expression.

Output: all `EVar` names appearing.

Why: fallback for variant/progress analysis when the variant is not in a simple recognized form.

post_write_bases : `sl` $\rightarrow$ `StringSet.t`. **Input:** SL formula.

Output: bases that appear in explicit post heap writes (e.g., `array[i]'`).

Why: identifies which array bases are mutated and should appear in assigns.

written_bases_from_ens_default_post : `sl` $\rightarrow$ `StringSet.t`. **Input:** SL formula (ensures).

Output: bases that are treated as post-state heap mentions by default unless protected under `EOld`.

Why: some ensures forms omit explicit `EPost` but semantically refer to post-state heap; this conservative analysis ensures assigns covers those bases.

6.14 Module Build

`mk_valid : string → Core.predicate.` **Input:** pointer variable name.

Output: Core predicate `valid(p)`.

Why: validity is a first-class precondition derived from heaplets.

`mk_valid_read_range / mk_valid_range : term → term → term → predicate.`

Input: base, low, high terms.

Output: Core predicates for read-only or mutable range validity.

Why: SL distinguishes read-only segments (`In`) from mutable (`Default`), which affects preconditions and frame.

`requires_from_ptrs : StringSet.t → predicate.` **Input:** set of pointers.

Output: conjunction of `valid(p)`.

Why: standardizes pointer validity generation.

`requires_from_ranges : spec_kind → range_atom list → predicate.` **Input:** contract kind and extracted range atoms.

Output: conjunction of range validity predicates (read vs write).

Why: range bounds may use expressions; converting them uses `Expr.term_of_expr` with correct phasing.

`progress_vars_from_variant : expr → StringSet.t.` **Input:** variant expression.

Output: the variable(s) treated as progress variables (e.g., from `bound - i`).

Why: loop invariant and assigns inference frequently needs the loop index variable; this provides a robust heuristic.

6.15 Module Ptrs

`ptrs_of_behavior : Sl_ast.behavior → StringSet.t.` **Input:** an SL behavior.

Output: set of all pointer locations relevant to the behavior.

Why: pointers can occur in: (1) explicit heaplets in `requires/ensures`, (2) pure heap equalities (swap sugar). This ensures `requires \valid` and heap assigns are not missed.

`global_ptrs_of_spec : Sl_ast.spec → StringSet.t.` **Input:** entire spec.

Output: union of pointers across all behaviors.

Why: when multiple behaviors exist, the translator may choose a shared pointer set so global `requires/assigns` are consistent.

6.16 Module SepNeq

This module infers pointer inequality constraints from SL separation (SSep) while respecting explicit inequalities.

`canon_pair` : `string` $\rightarrow$ `string` $\rightarrow$ (`string` * `string`). **Input:** two pointer names.

Output: ordered pair in canonical lexicographic order.

Why: ensures (p,q) and (q,p) are treated identically in sets.

`collect_explicit_neq_pairs` : `sl` $\rightarrow$ `PairSet.t`. **Input:** SL formula (typically requires).

Output: all explicitly stated `a != b` pairs.

Why: inferred constraints should not duplicate or contradict user-provided ones.

`collect_sep_loc_vars_in` : `sl` $\rightarrow$ `StringSet.t` list. **Input:** SL formula.

Output: list of pointer-variable sets, one per top-level separation group.

Why: SSep establishes disjointness among heaplets in the same group; extracting their locations allows generating pairwise inequalities.

`pairwise_neq_pairs` : `StringSet.t` $\rightarrow$ `PairSet.t`. **Input:** set of pointer names.

Output: all unordered pairs among them, canonicalized.

Why: separation implies all distinct locations are unequal.

`neq_predicate_from_pairs` : `PairSet.t` $\rightarrow$ `Core.predicate`. **Input:** set of inequality pairs.

Output: conjunction of `Core ptr(a) != ptr(b)` predicates.

Why: downstream requires construction expects a `Core` predicate.

`infer_sep_neqs` : `sl` $\rightarrow$ `Core.predicate`. **Input:** requires SL formula.

Output: inferred inequality predicate excluding explicitly stated pairs.

Why: implements the core semantic bridge: SL separation becomes explicit pointer inequality constraints.

6.17 Module Ensures

`ensures_from_post_heaplets` : `spec_kind` $\rightarrow$ `pt_atom.any` list $\rightarrow$ `predicate`.

Input: kind and post heaplets `p  $\rightarrow$  *(expr)`.

Output: conjunction of equalities `Heap(Post,p) == term(expr)`.

Why: explicit post heaplets are the most direct encoding of post-state heap updates.

ensures_from_pure_heap_eqs : (string*string) list $\rightarrow$ predicate. **Input:** extracted heap equalities from pure sugar.
Output: conjunction $\text{Heap}(\text{Post}, a) == \text{Heap}(\text{Pre}, b)$.
Why: swap-style specs express updates as cross-phase heap equalities; this normalizes them into heap-level ensures.

build_ensures : kind $\rightarrow$ post_sl $\rightarrow$ predicate. **Input:** kind and rewritten post SL formula.

Output: conjunction of:

- heaplet part (if any),
- pure heap equality part (if any),
- otherwise a general pure predicate translation for non-heap updates.

Why: ensures generation must cover three mutually exclusive “styles” used in tests: post heaplets, pure heap equalities, or purely logical postconditions.

6.18 Module SpecInfo

kind_of_spec : SL_last.spec $\rightarrow$ Core.spec_kind. **Input:** an SL spec with behaviors.

Output: LoopContract if any behavior contains a variant, else FunctionContract.

Why: loop contracts and function contracts differ structurally in Core: loops emit invariants/assigns/variant, while functions emit requires/assigns/ensures (and optional variant for termination).

normalize_behavior_names : behavior list $\rightarrow$ string option list. **Input:** list of behaviors.

Output: list of names where unnamed behaviors are assigned case# when there are multiple behaviors.

Why: ACSL behaviors must be named for readability and determinism; single-behavior specs stay unnamed to match expectations.

6.19 Top-level analysis: beh_analysis and analyze_behavior

Type beh_analysis. **Purpose:** a fully normalized, per-behavior summary used to build Core clauses:

- original and rewritten SL fragments (req_sl, ens_sl, post_sl_opt),
- Core predicates for assumes, requires-pure, inferred separation inequalities, ensures,
- extracted heap atoms, pre-map, and ranges,
- pointer set chosen for this behavior.

Why: downstream building becomes a pure assembly step: no repeated traversal, no duplicated heuristics.

`analyze_behavior` : `kind` $\rightarrow$ `spec_ret` $\rightarrow$ `ptrs_choice` $\rightarrow$ `behavior` $\rightarrow$ `beh_analysis`.

Input:

- `kind`: function vs loop contract,
- `spec_ret`: optional result variable name,
- `ptrs_choice`: local vs global pointer-set policy,
- an SL behavior `b` (assumes + block).

Output: `beh_analysis` record.

Why (pipeline rationale):

1. **Alias collection and expansion.** Aliases are collected from requires/ensures/assumes, unioned, and expanded with `n`-prefix conventions. This enforces a uniform base name for arrays across the behavior.
2. **Desugaring.** Alias-driven dereferences are rewritten to explicit `E0ld`/`EPost` reads.
3. **Heap source selection.** Heap information may appear in requires or assumes; the analysis picks a suitable source for extracting point-to atoms and ranges.
4. **Value-variable rewrite.** Using the pre heaplets, pure mentions of symbolic values are rewritten back into dereferences so that comparisons become heap comparisons (`a>=b` becomes `*p >= *q`).
5. **Separation inference.** From the requires, separation groups generate inferred `p != q` predicates.
6. **Pure requires predicate.** For loop contracts, pure constraints may be interpreted at post-phase; for function contracts, at pre-phase.
7. **Postcondition normalization.** Result rewriting, desugaring, old-forcing when post heap writes exist, and value-variable rewrites ensure ensures is in a canonical form before Core predicate construction.
8. **Pointer-set selection.** For multi-behavior specs, a shared pointer set avoids mismatched global requires/assigns; otherwise per-behavior pointers keep contracts minimal.

6.20 Module `Loop_contract`

This module derives loop invariants, assigns, and variants for loop contracts.

`scalar_accumulator_invariants` : `post_sl_opt` $\rightarrow$ `var_expr` $\rightarrow$ `predicate list`. **Input:** optional post SL and optional variant expression.
Output: inferred accumulator invariants of the form

$$x = x@LoopEntry + (i - i@LoopEntry)$$

when it detects the postcondition pattern $x' == x + (i' - i)$.

Why: this captures a common “for-loop accumulator” idiom used in the tests; emitting it as an invariant enables proof of arithmetic postconditions over loops.

`base_invariants` : `req_sep_neqs` $\rightarrow$ `ptrs` $\rightarrow$ `req_ranges` $\rightarrow$ `req_pure` $\rightarrow$ `predicate list`. **Input:** inferred separation inequalities, pointers, extracted ranges, and the pure predicate.

Output: a list beginning with the global conjunction, plus range-derived bounds (e.g., $0 \leq i \leq hi+1$) when the low bound is a variable.

Why: loop invariants must include: (1) stable global constraints (separation + pure), and (2) the standard bounds that arise from iterating over a segment.

`variant_clause` : `expr option` $\rightarrow$ `clause list`. **Input:** optional variant expression.

Output: empty or a singleton `Variant(term)` clause.

Why: loop termination is optional; this provides a uniform constructor.

`assigns_clause` : `req_ranges` $\rightarrow$ `post_sl_opt` $\rightarrow$ `var_expr` $\rightarrow$ `clause`. **Input:** ranges, optional post SL, optional variant.

Output: a single `Core Assigns` clause combining:

- scalar writes (progress variables and primed scalars),
- range writes for mutated bases (excluding `In` read-only ranges),
- with loop-entry lower bounds when the range lower bound coincides with the progress variable.

Why: loop assigns must precisely describe the loop frame: the index variable is always assigned, and arrays are assigned only when the postcondition indicates mutation. The special loop-entry lower-bound matches the typical “processed prefix / untouched suffix” framing.

`unchanged_suffix_invariants` : `req_ranges` $\rightarrow$ `var_expr` $\rightarrow$ `predicate list`. **Input:** ranges and variant.

Output: a quantified invariant stating that elements in the suffix $i \leq j < hi+1$ are unchanged from loop entry.

Why: this is the key preservation invariant used in array-processing loops: the loop only changes the prefix.

`processed_prefix_invariants : ensures_p → var_expr → predicate list.`

Input: ensures predicate and variant.

Output: invariants derived by rewriting ensures' quantified implications to apply to the processed prefix `i@LoopEntry <= j < i`.

Why: when ensures encodes per-element postconditions guarded by membership in the segment, the loop invariant should assert the property for the portion already processed. This routine extracts those implications and rewrites their guards accordingly.

`progress_invariants : req_ranges → var_expr → predicate list. In-`

put: ranges and variant.

Output: `i@LoopEntry <= i` when a single progress variable `i` is recognized.

Why: establishes monotonic progress, used by both suffix and prefix reasoning.

`build_invariants : ... → predicate list. Input:` all components needed for invariant inference.

Output: concatenation of base, progress, unchanged-suffix, processed-prefix, and scalar-accumulator invariants.

Why: centralizes invariant assembly in a single deterministic order to match expected printing and tests.

6.21 Clause builders: `mk_requires` and `mk_assigns`

`mk_requires : kind → ptrs → ranges → sep_neqs → pure → clause.`

Input: kind, pointer set, range atoms, inferred `p!=q` predicate, and pure predicate.

Output: a Core **Requires** clause containing their conjunction.

Why: requires is the single place where all pre-state obligations are accumulated: separation, validity, range validity, and pure constraints.

`mk_assigns : kind → ptrs → req_ranges → post_sl_opt → clause. In-`

put: kind, pointer set, range atoms, and optional post SL.

Output: Core **Assigns** clause listing mutated ranges and mutated scalar heap locations not covered by ranges.

Why: avoids double-counting: if an array base appears as a range assignment, the base is not also emitted as a heap-cell assignment. The written-base analysis combines explicit post-writes and implicit post heap mentions.

6.22 Behavior construction

`build_core_behavior : kind → name → beh_analysis → Core.behavior.`

Input: contract kind, optional behavior name, and `beh_analysis`.

Output: a Core behavior with appropriate clause layout:

- For function contracts: **Assumes**, **Requires**, **Ensures**, **Assigns**, plus optional **Variant**.

- For loop contracts: invariants emitted as **Assumes** clauses, then **Ensures**, **Assigns**, plus **Variant**.

Why: Core represents loop invariants uniformly as assumptions within the loop contract body, simplifying subsequent translation to ACSL loop-annotation form.

behavior_of_sl : `kind` $\rightarrow$ `spec_ret` $\rightarrow$ `name` $\rightarrow$ `ptrs_choice` $\rightarrow$ `behavior` $\rightarrow$ `Core.behavior`. **Input:** kind, optional result variable, name, pointer-choice policy, and SL behavior.

Output: Core behavior.

Why: this is the public per-behavior entry point: it runs `analyze_behavior` then assembles clauses.

6.23 Entry point: `sl_to_core`

sl_to_core : `SL_ast.spec` $\rightarrow$ `Core.spec`. **Input:** an SL specification consisting of zero or more behaviors and an optional result binder.

Output: a Core specification with:

- kind inferred by `SpecInfo.kind_of_spec`,
- normalized behavior names from `SpecInfo.normalize_behavior_names`,
- behaviors translated via `behavior_of_sl`.

Why:

- **Kind inference** determines the contract layout (function vs loop).
- **Name normalization** ensures deterministic behavior naming (important for multi-case ACSL behaviors).
- **Pointer-set policy:** when there are multiple behaviors, using `GlobalShared` pointers makes requires/assigns consistent across cases; otherwise `LocalPerBehavior` keeps contracts minimal.

7 Developer Documentation: `core_to_acsl`

This module translates the normalized Core AST (`Core`) into an ACSL AST (`Acs1_ast`), and finally prints it using `Acs1_ast_printer`. The key design goal is to: (1) preserve Core’s explicit phase semantics (`Pre`, `Post`, `LoopEntry`) and (2) render them into the correct ACSL surface forms depending on context (requires vs ensures vs loop invariant).

7.1 Context classification: ctx

type ctx = CRequires | CEnsures | CLoop | CLoopRel. **Meaning:**

- **CRequires:** printing terms/predicates in a function **requires** clause (pre-state by default).
- **CEnsures:** printing terms/predicates in a function **ensures** clause (mix of pre and post).
- **CLoop:** printing loop invariants (state typically at the current loop iteration, with explicit **LoopEntry** where needed).
- **CLoopRel:** printing relational invariants lifted from ensures (pre/post relations), normalized to **LoopEntry** vs current.

Why: Core terms carry explicit phases. ACSL surface syntax differs by clause: `\old(e)` and `\at(e, LoopEntry)` are only needed in certain contexts to refer to older states. `ctx` drives that decision centrally in `expr_of_core` and `pred_of_core`.

7.2 Operator mapping

`binop_of_rel : Core.rel -> Acsl.ast.binop.` **Input:** a Core relational operator (`Eq`, `Neq`, `Lt`, `Lte`, `Gt`, `Gte`).

Output: the corresponding ACSL binary operator node.

Why: Core keeps relations separate from arithmetic; ACSL still represents them as binary comparisons in the AST.

`binop_of_arith : Core.arith_op -> Acsl.ast.binop.` **Input:** a Core arithmetic operator (`Add`, `Sub`, `Mul`, `Div`).

Output: the corresponding ACSL arithmetic binop.

Why: ensures arithmetic expressions print using standard ACSL expression nodes.

7.3 Term normalization: simplify_term

`simplify_term : Core.term -> Core.term.` **Input:** a Core term.

Output: a semantically equivalent Core term with local algebraic simplifications.

Why: This is a printing-time simplifier that reduces noisy arithmetic artifacts introduced by earlier desugaring or inference. It improves readability and also helps matching against expected strings in tests.

Base cases. **Input:** `TInt`, `TResult`, `TPtr`, `TVar`, `Theap`.

Output: unchanged.

Why: These are already in simplest form.

Structural recursion. **Input:** TApp, TIndex, TLoad.

Output: same constructor with recursively simplified subterms.

Why: Simplification should be applied throughout the tree before arithmetic folding rules fire.

Arithmetic simplification: TArith(op,a,b). **Input:** an arithmetic node.

Output: simplified arithmetic node or simplified subterm.

Why: Common patterns occur frequently in inferred bounds, index arithmetic, and loop variants.

`strip_sub_const : term -> (term * int).` **Input:** a term.

Output: (x,c) where t is decomposed as $x - c$ for chained subtractions by constants.

Why: Enables cancelling patterns like $(x - k) + k$ even if nested, which otherwise prints with redundant offsets.

`fold_int : (int->int->int) -> term.` **Input:** a binary integer operator.

Output: constant-folded TInt when both operands are TInt, else leaves TArith.

Why: Small constant arithmetic appears in invariants and should be reduced early.

Rules for Add. Examples of transformations:

- $x + 0 \rightarrow x, 0 + x \rightarrow x$
- $(x - k) + k \rightarrow x$ (also commuted)
- generalized cancellation via `strip_sub_const`

Why: Loop bounds and range endpoints often produce this shape after desugaring.

Rules for Sub. Examples of transformations:

- $x - 0 \rightarrow x$
- $(x + k) - k \rightarrow x$
- constant fold $k1 - k2$

Why: Variants frequently take the form `bound - i`; these rules avoid clutter.

Rules for Mul. Examples of transformations:

- $x * 0 \rightarrow 0$
- $x * 1 \rightarrow x$
- constant folding $k1 * k2$

Why: Keeps arithmetic minimal and stable for printing.

Rules for Div. Examples of transformations:

- constant folding when denominator is nonzero

Why: Safe local simplification for common constant computations.

7.4 Term translation: `expr_of_core`

`expr_of_core` : `ctx` $\rightarrow$ `Core.term` $\rightarrow$ `Acsl.ast.expr`. **Input:** printing context `ctx` and a `Core` term.

Output: ACSL expression AST node.

Why: This function is the central phase-to-ACSL mapping layer. It:

- calls `simplify_term` first to reduce noise,
- then emits `\old` or `\at(..., LoopEntry)` depending on `ctx` and the term's phase.

Constants and base terms. **Input:** `TInt`, `TResult`, `TPtr`.

Output: `EConstInt`, `EResult`, `EVar`.

Why: Pointers are printed as variables in expressions; dereferencing is explicit via `THeap/TLoad` paths.

Scalar variables: `TVar(ph,x)`. **Input:** a phased variable.

Output: depends on `ctx`:

- `CRequires`: always `EVar x` (requires is pre-state).
- `CEnsures`: `Post` $\rightarrow$ `EVar x`, `Pre` $\rightarrow$ `\old(x)`, `LoopEntry` $\rightarrow$ `\at(x, LoopEntry)`.
- `CLoop`: `LoopEntry` $\rightarrow$ `\at(x, LoopEntry)`, otherwise `EVar x`.
- `CLoopRel`: `Post` $\rightarrow$ `EVar x`, `Pre/LoopEntry` $\rightarrow$ `\at(x, LoopEntry)`.

Why:

- In function ensures, pre-state must be made explicit using `\old`.
- In loop invariants, `LoopEntry` is the stable reference point.
- For lifted relational invariants (`CLoopRel`), we intentionally collapse both `Pre` and `LoopEntry` into `\at(..., LoopEntry)` so the relational constraint reads “LoopEntry vs current”.

Heap cell reads: `THeap(ph,p)`. **Input:** a phased heap cell read at pointer `p`.

Output: builds `EDeref(EVar p)` and then wraps by context:

- `CRequires`: plain `deref` (pre-state).

- **CEnsures:** $\text{Pre} \rightarrow \backslash\text{old}(*p), \text{Post} \rightarrow *p, \text{LoopEntry} \rightarrow \backslash\text{at}(*p, \text{LoopEntry})$.
- **CLoop:** $\text{LoopEntry} \rightarrow \backslash\text{at}(*p, \text{LoopEntry})$, else $*p$.
- **CLoopRel:** $\text{Pre}/\text{LoopEntry} \rightarrow \backslash\text{at}(*p, \text{LoopEntry}), \text{Post} \rightarrow *p$.

Why: Same reasoning as scalar variables, but heap reads must be explicit dereferences.

Unary negation canonical form. **Input:** $\text{TArith}(\text{Sub}, \text{TInt } 0, t)$.

Output: $\text{EUnop}(\text{UNeg}, \text{expr_of_core } t)$.

Why: Prints $-t$ instead of $0 - t$, improving readability.

General arithmetic, applications. **Input:** $\text{TArith}(\text{op}, t_1, t_2)$ and $\text{TApp}(f, \text{args})$.

Output: EBinop or EApp .

Why: Direct structural translation after phase-correct subterm translation.

Array indexing: $\text{TIndex}(\text{ph}, a, \text{idx})$. **Input:** phased indexing term.

Output: $\text{EIndex}(a', \text{idx}')$ possibly wrapped in $\backslash\text{old}$ or $\backslash\text{at}$.

Why (special handling):

- In **CEnsures** with $\text{ph}=\text{Pre}$, the code emits $\backslash\text{old}(a[\text{idx}])$. To avoid double-olding when a or idx contain $\text{TVar } \text{Pre}$, it first coerces those TVar to Post before printing them inside $\backslash\text{old}$. This yields $\backslash\text{old}(\text{array}[j])$ instead of $\backslash\text{old}(\backslash\text{old}(\text{array})[\backslash\text{old}(j)])$.
- In **CLoopRel**, $\text{Pre}/\text{LoopEntry}$ index expressions become $\backslash\text{at}(a[\text{idx}], \text{LoopEntry})$, while Post stays unwrapped.
- In **CRequires/CLoop**, LoopEntry becomes $\backslash\text{at}(\dots, \text{LoopEntry})$, otherwise unwrapped.

Why this exists: indexing is the most common source of “nested old” printing bugs; the pre-index rule is an explicit guardrail.

General loads: $\text{TLoad}(\text{ph}, \text{addr})$. **Input:** load term.

Output: translates addr , forms $\text{EDeref}(\text{addr})$, then wraps by ctx similar to THeap .

Why: TLoad represents dereference of a non-variable address; it still needs correct phase printing.

7.5 Type translation: `sort_of_core_ty_opt`

`sort_of_core_ty_opt` : `string option -> AcsLast.sort option`. **Input:** optional Core type string.

Output: optional ACSL sort.

Why: Core stores binder type annotations as strings; ACSL AST expects a structured sort. This function normalizes common synonyms (`int/integer`, `bool/boolean`) and allows user sorts.

7.6 Predicate translation: `pred_of_core`

`pred_of_core` : `ctx -> Core.predicate -> Acsl_ast.pred`. **Input:** context and Core predicate.

Output: ACSL predicate AST.

Why: Converts logical structure and special predicate symbols (validity predicates) into ACSL-native constructs.

Truth constants. **Input:** `PTrue`, `PFalse`.

Output: `A.PTrue`, `A.PFalse`.

Why: direct.

Relational atoms: `PAtom(ARel(r,t1,t2))`. **Input:** relation and two terms.

Output: `PCmp(binop, expr1, expr2)`.

Why: ACSL represents comparisons at predicate level.

Named predicate atoms: `PAtom(APred(name,args))`. **Input:** predicate name and arguments.

Output: specialized ACSL nodes for validity predicates, else `PApp`.

Why: `\valid` and range-validity have dedicated ACSL syntax, so the module canonicalizes them:

- "valid":
 - `[TPtr v] -> PValid(EVar v)`
 - `[t] -> PValid(expr(t))`
 - otherwise fallback to `PApp("valid", ...)`
- "valid_read_range" and "valid_range":
 - expects `[base; lo; hi]` and emits `\valid_read(base + (lo..hi))` or `\valid(base + (lo..hi))` by constructing `EBinop(BAdd, base, ERange(lo,hi))`.
 - otherwise falls back to `PApp` with `\valid_read` or `\valid`.

This keeps output idiomatic ACSL and reduces printer complexity.

Logical connectives and quantifiers. **Input:** `PNot`, `PAnd`, `POr`, `PImplies`, `PForall`, `PExists`.

Output: corresponding ACSL predicate nodes, with binders mapped using `sort_of_core_ty_opt`.

Why: direct structure preservation.

7.7 Helper utilities

These helpers serve two main purposes: (1) generic list extraction/normalization, (2) deterministic de-duplication of predicates and assigns targets.

find_first : ('a -> 'b option) -> 'a list -> 'b option. **Input:** extractor and list.

Output: first extracted **Some** value, or **None**.

Why: clauses like **Requires** or **Assigns** are expected at most once in chosen contexts.

all_of : ('a -> 'b option) -> 'a list -> 'b list. **Input:** extractor and list.

Output: all extracted values.

Why: **Assumes** and **Ensures** may appear multiple times (especially in loop contracts where invariants are emitted as assumes).

clause_assumes / **clause_requires** / **clause_ensures** / **clause_assigns**. **Input:** Core clause.

Output: optional payload if the clause matches.

Why: local pattern helpers to make higher-level code readable.

split_top_and_core : Core.predicate -> Core.predicate list. **Input:** a Core predicate.

Output: list of conjuncts if the predicate is a top-level **PAnd**, otherwise singleton.

Why: normalization steps want to treat **PAnd** lists as a flat multiset for deduplication.

uniq_core_preds : Core.predicate list -> Core.predicate list. **Input:** predicate list.

Output: sorted unique list (using **Stdlib.compare**).

Why: deterministic predicate combination for expected-output stability.

pred_and_core : Core.predicate list -> Core.predicate. **Input:** list of Core predicates.

Output: canonical conjunction with: flattened top-level **PAnd**, **PTrue** removed, duplicates removed.

Why: used when aggregating global requires/ensures across behaviors.

uniq_preserve : 'a list -> 'a list. **Input:** list.

Output: first-occurrence unique list preserving order.

Why: ACSL output is order-sensitive for readability and tests; this provides stable de-dup without sorting.

normalize_pred_list : A.pred list -> A.pred list. **Input:** ACSL predicate list.

Output: list with **PTrue** removed and duplicates removed preserving order.

Why: avoids emitting redundant invariants/assumes when multiple sources contribute.

pred_and_acsl : A.pred list -> A.pred. **Input:** ACSL predicates.
Output: canonical PAnd with nested PAnd flattened and duplicates removed.
Why: ensures clean printing in ACSL AST.

uniq_assignables : Core.assignable list -> Core.assignable list. **Input:** Core assignables list.
Output: de-duplicated list by a string key: AsVar, AsHeap, AsRange keyed by base name, AsTerm keyed by hash.
Why: global assigns is formed by union across behaviors; duplicates are expected and should not be printed twice.

7.8 Assigns translation

assigns_target_of_core : Core.assignable -> A.assigns_target option.
Input: a Core assignable.
Output: optional ACSL assigns target:

- AsVar v → AVar v
- AsHeap p → ADeref(EVar p)
- AsRange(p, lo, hi) → ARange(EVar p, lo', hi') (printed in CLoop context)
- AsTerm t → ADeref(expr(t)) (printed in CEnsures context)

Why: ACSL assigns requires a restricted target language; Core's assignables are normalized into those targets.

assigns_of_core : Core.assignable list -> A.assigns. **Input:** list of assignables.
Output: ANothing if empty, else AItems.
Why: ACSL requires an explicit \nothing when empty.

7.9 Function contract emission: fun_spec_of_core

is_global_req_only_behavior : Core.behavior -> bool. **Input:** a behavior.
Output: true if the behavior contains only a nontrivial requires and nothing else.
Why: This identifies artificial behaviors whose only purpose was to contribute to global requires aggregation. Such behaviors should not be printed as ACSL behaviors.

`fun_spec_of_core : Core.spec -> A.fun_spec.` **Input:** a Core function spec.

Output: an ACSL function spec with:

- **requires:** conjunction of all Core requires across behaviors,
- **assigns:** union of all assigns across behaviors,
- **behaviors:** printed only if there exists at least one named behavior,
- **ensures:** printed globally only if behaviors are not printed.

Why (design):

- Core may have per-behavior requires, but ACSL typically places requires globally; hence aggregation.
- If behaviors are used, per-behavior assumes/ensures are printed under those behaviors, and global ensures is omitted to avoid duplication.
- **complete** and **disjoint** are emitted when there are multiple named behaviors, matching ACSL behavior semantics.

7.10 Relational lifting for loops

The loop pipeline optionally lifts certain relational ensures predicates into invariants.

`term_mentions_result / pred_mentions_result.` **Input:** term/predicate.

Output: whether

result appears.

Why: relational invariants should not reference result in loop context.

`term_has_phase / pred_has_phase.` **Input:** phase and term/predicate.

Output: whether that phase appears.

Why: used to detect pre/post-relational structure.

`pred_is_relational_pre_post : predicate -> bool.` **Input:** predicate.

Output: true if predicate mentions both **Pre** and **Post**.

Why: relational invariants are precisely those connecting loop-entry (pre) and current (post).

`is_liftable_relational : predicate -> bool.` **Input:** predicate.

Output: true if the predicate is an equality relation between pre/post terms, with allowances for **forall** and **implies** wrappers and conjunctions, and excluding result.

Why: The lifting step is conservative: it only lifts patterns that correspond to stable “frame”-like equalities (e.g., unchanged segments) that are safe and useful as invariants.

7.11 Loop contract emission: `loop_spec_of_core`

`loop_spec_of_core` : `Core.spec -> A.loop_spec`. **Input:** a Core loop spec.
Output: ACSL loop spec:

- chooses one behavior (prefer one containing a variant),
- invariants = assumes-clauses translated under `CLoop` plus lifted relational ensures under `CLoopRel`,
- assigns = the chosen behavior’s assigns,
- variant = the chosen behavior’s variant (if present).

Why:

- Loop annotations in ACSL are single blocks; Core may carry multiple behaviors, but loop translation expects a single chosen set.
- Assumes in Core loop contracts represent invariants inferred upstream.
- Lifted relational ensures convert postconditions of the form `pre==post` into invariants relative to `LoopEntry`, matching common verification patterns (`suffix unchanged`).

7.12 Entry point: `spec_to_acsl`

`spec_to_acsl` : `Core.spec -> string`. **Input:** a Core spec.

Output: rendered ACSL string.

Why: This is the public API. It selects:

- `FunSpec` for `FunctionContract`,
- `LoopSpec` for `LoopContract`,

and delegates printing to `Acs1_ast_printer.string_of_spec`. This keeps all formatting logic confined to the ACSL printer layer.

8 Unit Tests Coverage

The unit tests serve as executable documentation of the translator testing for end-to-end behavior. Below is a categorized summary of how each group of tests is translated.

8.1 Swap and Heap Equality Tests

Tests such as `swap`, `triple_swap` and their notation variants verify that:

- Point-to assertions in `ens` are converted into heap equalities.
- Prime and `\old` notations are normalized into the same Core representation.
- Assigns clauses correctly include all modified heap locations.

8.2 Pure Expression Tests

Unary and arithmetic tests ensure that:

- Logical negation and arithmetic negation are preserved.
- No spurious heap effects are inferred.
- `assigns \nothing` is emitted when appropriate.

8.3 Case and Behavior Tests

Single- and multi-case tests validate that:

- Each case becomes a Core behavior with an `assumes` clause.
- Common `requires` and `assigns` are lifted globally.
- `complete` and `disjoint` behaviors are emitted when needed.

8.4 Loop and Termination Tests

Termination-based tests check that:

- `Term[e]` becomes a loop variant.
- Guards become loop invariants.
- Modified variables and ranges are inferred into loop assigns.

8.5 Array and Range Tests

Array-focused tests ensure that:

- Range heaplets produce `valid` or `valid_read` requirements.
- Aliased notations (`arr`, `narr`) normalize to the base array.
- Writes to array elements infer appropriate range assigns.

8.6 Search and Replace Tests

Search-style and search-replace tests validate:

- Quantified invariants are preserved.
- Loop-entry values are correctly referenced using `\at(..., LoopEntry)`.
- Combined heap and pure reasoning is handled consistently.

8.7 Result-based Tests

Tests involving `ens[r]` ensure that:

- Return variables are rewritten to `\result`.